

Property-based testing with fast-check

How we test the GeoJSON factories. State what must always be true, let fast-check search for inputs that break it, and pin every bug it finds. All examples come from this repo and run in npm test.

01 The idea

An example test checks one input that you picked. A **property** is a rule that must hold for every valid input, such as "a normalised bearing is always in [0, 360) and points the same way". fast-check then tries to break it: by default it generates 100 inputs per run.

docs/examples/fast-check/01-the-idea.test.ts

```
// Example-based: we choose the inputs, so we only test the cases we thought of.
describe('normaliseBearing: examples', () => {
  it.each([[[-90, 270], [360, 0], [725, 5]]])('%i° -> %i°', (input, expected) => {
    expect(normaliseBearing(input)).toBe(expected);
  });
});

// Property-based: fast-check chooses the inputs; we state what must hold for all of them.
describe('normaliseBearing: properties', () => {
  const arbAnyBearing = fc.double({ min: -1e6, max: 1e6, noNaN: true });
  const rad = (deg: number) => (deg * Math.PI) / 180;

  test.prop([arbAnyBearing])('lands in [0, 360)', (deg) => {
    const b = normaliseBearing(deg);
    expect(b).toBeGreaterThanOrEqual(0);
    expect(b).toBeLessThan(360);
  });

  test.prop([arbAnyBearing])('still points the same way', (deg) => {
    const b = normaliseBearing(deg);
    expect(Math.cos(rad(b))).toBeCloseTo(Math.cos(rad(deg)), 6);
    expect(Math.sin(rad(b))).toBeCloseTo(Math.sin(rad(deg)), 6);
  });
});
```

The examples document intent. The properties catch the cases nobody wrote down. We keep both. fast-check does four things that a loop over `Math.random()` doesn't:

Concept	What it means for you
Generation	You describe the <i>shape</i> of the input with an arbitrary (\$03). fast-check builds values from it.
Bias	Bounds, 0, -0 and tiny numbers come up far more often than chance. This is where our bugs were.

Concept	What it means for you
Shrinking	After a failure, fast-check looks for a <i>smaller</i> input that still fails and reports that one.
Seeds	Every run comes from one seed. The same seed gives the same inputs, so any failure can be replayed.

Bias, in numbers: out of 1,000 latitudes from `fc.double({ min: -90, max: 90 })`, the run below contains both bounds, negative zero and values smaller than $1e-300$:

`docs/examples/fast-check/01-the-idea.test.ts`

```
it('biases generation toward edge cases', () => {
  const arbLat = fc.double({ min: -90, max: 90, noNaN: true });
  const lats = fc.sample(arbLat, { seed: 1, numRuns: 1000 });
  expect(lats).toContain(-90); // both bounds
  expect(lats).toContain(90);
  expect(lats.some((x) => Object.is(x, -0))).toBe(true); // negative zero
  expect(lats.some((x) => x !== 0 && Math.abs(x) < 1e-300)).toBe(true); // tiny values
});
```

Shrinking turns the first failure fast-check stumbled on (here `-428`) into the simplest one (`-1`), which is the one you want to debug:

`docs/examples/fast-check/01-the-idea.test.ts`

```
it('shrinks a failure to a minimal counterexample', () => {
  const naiveNormalise = (deg: number) => deg % 360; // forgets negative bearings
  const details = fc.check(
    fc.property(fc.integer({ min: -720, max: 720 }), (deg) => naiveNormalise(deg) >= 0),
    { seed: 42, verbose: true },
  );
  expect(details.failed).toBe(true);
  expect(details.failures[0]).toEqual([-428]); // the first failing input it generated
  expect(details.counterexample).toEqual([-1]); // what it reports after shrinking
});
```

02 Setup with Vitest

```
npm install -D fast-check @fast-check/vitest
```

We use **fast-check 4.10** with **@fast-check/vitest 0.5** on Vitest 5. Import both `test` and `fc` from `@fast-check/vitest`. It re-exports the `fc` it runs with, so the global settings in `test/setup.ts` are the ones `test.prop` uses.

`test.prop` takes the arbitraries either as a tuple (one argument each) or as a record (one named object):

`docs/examples/fast-check/02-setup.test.ts`

```
// Tuple form: one arbitrary per argument.
test.prop([arbAisReport])('an AIS feature never has a position outside ±180/±90', (m) => {
  const f = aisToFeature(m);
  if (f === null) return; // no position: nothing to check
  const [lng, lat] = f.geometry.coordinates as [number, number];
```

```

    expect(Math.abs(lng)).toBeLessThanOrEqual(180);
    expect(Math.abs(lat)).toBeLessThanOrEqual(90);
  });

  // Record form: one named object, handy when there are several inputs.
  const arbCoverageInput = {
    station: arbSafePosition,
    rangeKm: fc.double({ min: 0.5, max: 500, noNaN: true }),
  };
  test.prop(arbCoverageInput)('coverage contains its station', ({ station, rangeKm }) => {
    expect(booleanPointInPolygon(station, coverageToFeature(station, rangeKm, 'rx'))).toBe(true);
  });

```

`test.prop` adds the seed to the test name, e.g. `coverage contains its station (with seed=-1359808129)`. That is the seed you need to replay a failure (§06). Inside a normal `it`, use `fc.assert(fc.property(...)).fc.pre(condition)` skips the current input instead of failing:

docs/examples/fast-check/02-setup.test.ts

```

// Plain fast-check inside a normal Vitest test: useful inside loops, helpers or it.each.
it('every report without a position becomes null', () => {
  fc.assert(
    fc.property(arbAisReport, (m) => {
      fc.pre(m.lat === 91 || m.lon === 181); // only reports with the "not available" sentinel
      expect(aisToFeature(m)).toBeNull();
    }),
  );
});

```

Parameters go in the second argument. These ones override the global settings for this test only:

docs/examples/fast-check/02-setup.test.ts

```

// Per-test parameters override the global ones from test/setup.ts.
test.prop([fc.array(arbAisReport, { maxLength: 200 })], { numRuns: 25 })(
  'a slow property over big batches: fewer runs',
  (reports) => {
    const features = reports.map(aisToFeature).filter((f) => f !== null);
    expect(features.length).toBeLessThanOrEqual(reports.length);
  },
);

```

Global settings live in `test/setup.ts`, which Vitest loads before every test file, including `docs/examples`:

test/setup.ts

```

import { fc } from '@fast-check/vitest';

// Default 100 runs per property. Hunt harder locally with: FC_RUNS=2000 npm test
// Replay a failure exactly with: FC_SEED=<seed from the failure message> npm test
fc.configureGlobal({
  numRuns: Number(process.env.FC_RUNS ?? 100),
  ...(process.env.FC_SEED ? { seed: Number(process.env.FC_SEED) } : {}),
});

```

vite.config.ts

```
test: {
  environment: 'node',
  // docs/examples holds every code snippet in the onboarding PDFs, so the guides stay tested
  include: ['test/**/*.test.ts', 'docs/examples/**/*.test.ts'],
  setupFiles: ['test/setup.ts'],
},
```

Command	What it does
<code>npm test</code>	100 runs per property, a new random seed every time
<code>FC_RUNS=2000 npm test</code>	Hunt harder before a release or after touching a factory
<code>FC_SEED=<seed> npm test</code>	Replay a failure exactly. Add <code>-t "<test name>"</code> to run just that test

Upgrading from fast-check 3? Version 4 changed some defaults. `fc.date()` now generates `Invalid Date` unless you pass `noInvalidDate: true` (this was the v3 default), and `toISOString()` throws on it. `fc.record()` and `fc.dictionary()` may now return objects with a null prototype (pass `noNullPrototype: true` if your code needs `Object.prototype`). `withDeletedKeys` is gone (use `requiredKeys: []`). So are `hexaString`, `stringOf`, `unicodeString`, `asciiString`, `char` and `ascii`; use `fc.string({ unit })` or `.map()` instead.

03 Arbitraries

An **arbitrary** (`fc.Arbitrary<T>`) knows how to generate values of type `T`, and how to shrink them. These are the building blocks we use:

Building block	Generates	Used for
<code>fc.integer({ min, max })</code>	Whole numbers in range, biased toward the bounds and 0	MMSI, heading, baro rate
<code>fc.double({ min, max, noNaN })</code>	64-bit floats, including <code>-0</code> and tiny values. NaN unless <code>noNaN</code> ; $\pm$ Infinity unless min and max are finite	lat, lng, SOG
<code>fc.constant(v)</code>	Always <code>v</code>	AIS sentinels: lat 91, heading 511
<code>fc.constantFrom(a, b, ...)</code>	One of the listed values	modes, call signs, categories
<code>fc.oneof({ weight, arbitrary }, ...)</code>	A value from one of several arbitraries, chosen by weight	18 real latitudes : 2 sentinels : 1 garbage
<code>fc.record({ ... }, { requiredKeys })</code>	Objects. Keys not in <code>requiredKeys</code> may be missing	readsb aircraft (Mode S has no lat/lon)

Building block	Generates	Used for
<code>fc.array(arb, { minLength, maxLength })</code>	Arrays between the length bounds (<code>fc.uniqueArray</code> for no duplicates)	track samples, report batches
<code>fc.tuple(a, b)</code>	Fixed-length arrays of mixed types	<code>[lng, lat]</code>
<code>fc.option(arb, { nil, freq })</code>	A value or <code>nil</code> (<code>null</code> by default, about 1 run in 6)	optional squawk
<code>arb.map(f)</code>	<code>f(value)</code> . Shrinks through the source value	ICAO24 hex from an integer
<code>arb.filter(pred)</code>	Only values that pass; the rest are thrown away	vessels that are moving
<code>arb.chain(f)</code>	A second arbitrary built from a generated value	a fleet, then reports for that fleet

The same blocks in code. Every line is checked by `03-arbitraries.test.ts`:

`docs/examples/fast-check/03-arbitraries.test.ts`

```
const arbMmsi = fc.integer({ min: 200_000_000, max: 799_999_999 });
const arbLat = fc.double({ min: -90, max: 90, noNaN: true }); // finite bounds: no ±Infinity
const arbHeadingNA = fc.constant(511); // AIS "heading not available"
const arbMode = fc.constantFrom('FT8', 'CW', 'SSB', 'WSPR');

// oneof: pick one arbitrary per value; weights set how often (here 9 : 1)
const arbHeading = fc.oneof(
  { weight: 9, arbitrary: fc.integer({ min: 0, max: 359 }) },
  { weight: 1, arbitrary: arbHeadingNA },
);

const arbPosition = fc.tuple(fc.double({ min: -180, max: 180, noNaN: true }), arbLat);
const arbSquawk = fc.option(fc.stringMatching(/^([0-7]{4})$/), { nil: undefined }); // or absent

// map: transform each value (and shrinking still works)
const arbIcao24 = fc
  .integer({ min: 0, max: 0xffffffff })
  .map((n) => n.toString(16).padStart(6, '0'));

// filter: reject values; fine when few are rejected
const arbMovingSog = fc.double({ min: 0, max: 102.2, noNaN: true }).filter((kn) => kn >= 0.5);

// record: only `hex` is always present, the other keys may be missing
const arbAircraft = fc.record(
  { hex: arbIcao24, lat: arbLat, squawk: arbSquawk },
  { requiredKeys: ['hex'] },
);

// array: between 0 and 30 positions
const arbTrack = fc.array(arbPosition, { maxLength: 30 });

// chain: the second arbitrary depends on a generated value.
// First a fleet of 1-5 MMSIs, then reports that only use those MMSIs (so they collide).
const arbFleetReports = fc
```

```
.uniqueArray(arbMmsi, { minLength: 1, maxLength: 5 })
.chain((fleet) =>
  fc.array(fc.record({ mmsi: fc.constantFrom(...fleet), heading: arbHeading })),
);
```

You never write a shrinker. Everything built from these blocks shrinks, including the results of `map` and `chain`. That is a good reason to build domain values from the blocks (as in §04) instead of generating them with your own random code.

04 Writing domain arbitraries

Our arbitraries live in `test/arbitraries.ts`. Every property in the repo uses them, so we only describe "what a real AIS report looks like" once.

Positions. Two versions: one covers the whole globe, and one stays away from the poles and the antimeridian for simple geometry models. Use the full one wherever you can, because the edges are where the bugs are:

`test/arbitraries.ts`

```
export const arbLng = fc.double({ min: -180, max: 180, noNaN: true });
export const arbLat = fc.double({ min: -90, max: 90, noNaN: true });
export const arbPosition: fc.Arbitrary<Position> = fc.tuple(arbLng, arbLat);

/** Away from the poles and the antimeridian, where simple geometry models behave. */
export const arbSafePosition: fc.Arbitrary<Position> = fc.tuple(
  fc.double({ min: -170, max: 170, noNaN: true }),
  fc.double({ min: -80, max: 80, noNaN: true }),
);

/** Values a broken decoder might emit. */
export const arbGarbageNumber = fc.constantFrom(
  Number.NaN, Number.POSITIVE_INFINITY, Number.NEGATIVE_INFINITY, 1e9, -1e9,
);
```

AIS, with sentinels. ITU-R M.1371 encodes "not available" as in-band values (latitude 91, longitude 181, SOG 102.3, COG 360, heading 511). Real feeds also contain values a broken decoder produced. Weighted `oneof` mixes both in at a known rate, so most reports are still realistic:

`test/arbitraries.ts`

```
/** AIS report with sentinels mixed in (~10%) and occasional decoder garbage. */
export const arbAisReport: fc.Arbitrary<AisPositionReport> = fc.record({
  mmsi: fc.integer({ min: 200_000_000, max: 799_999_999 }),
  lat: fc.oneof(
    { weight: 18, arbitrary: arbLat },
    { weight: 2, arbitrary: fc.constant(91) }, // "not available"
    { weight: 1, arbitrary: arbGarbageNumber },
  ),
  lon: fc.oneof(
    { weight: 18, arbitrary: arbLng },
    { weight: 2, arbitrary: fc.constant(181) }, // "not available"
    { weight: 1, arbitrary: arbGarbageNumber },
  ),
  sog: fc.oneof(
    { weight: 9, arbitrary: fc.double({ min: 0, max: 102.2, noNaN: true }) },
  ),
});
```

```

    { weight: 1, arbitrary: fc.constant(102.3) },
  ),
  cog: fc.oneof(
    { weight: 9, arbitrary: fc.double({ min: 0, max: 359.9, noNaN: true }) },
    { weight: 1, arbitrary: fc.constant(360) },
  ),
  heading: fc.oneof(
    { weight: 9, arbitrary: fc.integer({ min: 0, max: 359 }) },
    { weight: 1, arbitrary: fc.constant(511) },
  ),
  navStatus: fc.integer({ min: 0, max: 15 }),
  name: fc.string({ maxLength: 20 }),
  timestamp: arbIsoTime,
});

```

ICAO24 hex, built with `map`. We don't use a string arbitrary with a regex. We generate the 24-bit integer and format it. Every value is then valid by construction, and shrinking heads toward `000000`:

test/arbitraries.ts

```

/** ICAO24 as lowercase hex, optionally "~"-prefixed (non-ICAO / TIS-B). */
export const arbHex: fc.Arbitrary<string> = fc
  .tuple(fc.integer({ min: 0, max: 0xffffffff }), fc.boolean())
  .map(([n, tilde]) => (tilde ? '~' : '') + n.toString(16).padStart(6, '0'));

```

readsb aircraft. Only `hex` is always present. Everything else may be missing, which is exactly what `aircraft.json` looks like for a Mode S-only target:

test/arbitraries.ts

```

export const arbReadsbAircraft: fc.Arbitrary<ReadsbAircraft> = fc.record(
  {
    hex: arbHex,
    flight: fc.string({ maxLength: 8 }),
    alt_baro: fc.oneof(fc.integer({ min: -1000, max: 50000 }), fc.constant('ground' as const)),
    alt_geom: fc.integer({ min: -1000, max: 51000 }),
    gs: fc.double({ min: 0, max: 700, noNaN: true }),
    track: fc.double({ min: 0, max: 359.99, noNaN: true }),
    baro_rate: fc.integer({ min: -6000, max: 6000 }),
    squawk: fc.integer({ min: 0, max: 0o7777 }).map((n) => n.toString(8).padStart(4, '0')),
    category: fc.constantFrom('A1', 'A3', 'A5', 'A7', 'B2'),
    lat: arbLat,
    lon: arbLng,
    seen: fc.double({ min: 0, max: 300, noNaN: true }),
  },
  // lat/lon may be missing (Mode S only); everything else optional too
  { requiredKeys: ['hex'] },
);

```

Times, tracks and HF spots. `fc.date().map(toISOString)` gives the same ISO 8601 strings the feeds send. `noInvalidDate` matters in fast-check 4 (see the warning in §02):

test/arbitraries.ts

```

const arbIsoTime = fc
  .date({
    min: new Date('2020-01-01T00:00:00Z'),
    max: new Date('2030-01-01T00:00:00Z'),
  })
  .map(toISOString);

```

```

    noInvalidDate: true,
  })
  .map((d) => d.toISOString());

```

test/arbitraries.ts

```

export const arbTrackSamples: fc.Arbitrary<TrackSample[]> = fc.array(
  fc.record({ lon: arbLng, lat: arbLat, t: arbIsoTime }),
  { minLength: 0, maxLength: 60 },
);

```

test/arbitraries.ts

```

export const arbHfSpot: fc.Arbitrary<HfSpot> = fc.record({
  tx: arbPosition,
  rx: arbPosition,
  txCall: fc.constantFrom('DL1ABC', 'JA1AAA', 'VK2DEF'),
  rxCall: fc.constantFrom('W2XYZ', 'K6BBB', 'ZS1CCC'),
  frequencyKHz: fc.double({ min: 1800, max: 29700, noNaN: true }),
  mode: fc.constantFrom('FT8', 'CW', 'SSB', 'WSPR'),
  snrDb: fc.integer({ min: -30, max: 30 }),
  timestamp: arbIsoTime,
});

```

Test the generator too. If `arbAisReport` never produced latitude 91, every property about sentinels would pass without testing anything. `fc.sample` with a fixed seed lets you check the mix. While exploring, `fc.statistics(arb, classify)` prints the share of each class you define:

docs/examples/fast-check/04-domain-arbitraries.test.ts

```

it('arbAisReport mixes in sentinels and garbage at the rates we intended', () => {
  const reports = fc.sample(arbAisReport, { seed: 2026, numRuns: 2000 });
  const share = (pred: (m: (typeof reports)[number]) => boolean) =>
    reports.filter(pred).length / reports.length;

  expect(share((m) => m.lat === 91)).toBeGreaterThan(0.05); // weight 2 of 21 ≈ 9.5%
  expect(share((m) => m.lat === 91)).toBeLessThan(0.15);
  expect(share((m) => Number.isNaN(m.lat))).toBeGreaterThan(0); // decoder garbage occurs
  expect(share((m) => m.heading === 511)).toBeGreaterThan(0.05);
});

it('arbReadsbAircraft sometimes omits the position (Mode S only)', () => {
  const aircraft = fc.sample(arbReadsbAircraft, { seed: 2026, numRuns: 500 });
  expect(aircraft.some((a) => a.lat === undefined)).toBe(true);
  expect(aircraft.some((a) => a.lat !== undefined && a.lon !== undefined)).toBe(true);
});

```

05 Property patterns

Most properties fall into a few patterns. If you can't think of a property, go through this list:

Pattern	The question it asks	Our example
Validity	Is the output valid GeoJSON, or <code>null</code> exactly when it should be?	<code>aisToFeature</code>

Pattern	The question it asks	Our example
Preservation	Does the input survive, in <code>[lng, lat]</code> order?	<code>aisToFeature</code>
Counting	Is nothing lost and nothing invented?	<code>vesselsToCollection</code>
Geometric invariant	Does the shape mean what it should? Turf checks it	<code>coverageToFeature</code>
Structural rule	Can MapLibre draw it without artefacts?	<code>hfPathToFeature</code>
Idempotence	Does doing it twice change anything?	<code>round6</code>
Round-trip	Out and back again: do we get the input?	JSON, bearings
Order-independence	Does the order of the input matter?	tracks, vessels

Validity. The most valuable property, and the easiest to write: whatever goes in, the output is valid, or `null` exactly when the input has no position. `hasPosition` is a small helper at the top of the test file that spells out the AIS rules:

test/ais.test.ts

```
test.prop([arbAisReport])('always valid GeoJSON, or null exactly when there is no position',
(m) => {
  const f = aisToFeature(m);
  if (!hasPosition(m)) { expect(f).toBeNull(); return; }
  expect(f).not.toBeNull();
  expectValidGeoJSON(f);
},
);
```

Preservation. Catches swapped coordinates, which a validator can't catch when both values are within ± 90 :

test/ais.test.ts

```
test.prop([arbAisReport])('preserves the position in [lng, lat] order', (m) => {
  fc.pre(hasPosition(m));
  const [lng, lat] = aisToFeature(m)!.geometry.coordinates as [number, number];
  expect(lng).toBeCloseTo(m.lon, 5);
  expect(lat).toBeCloseTo(m.lat, 5);
});
```

Counting. One feature per MMSI that has a position, no more and no less:

test/ais.test.ts

```
const arbBatch = fc.array(arbAisReport, { maxLength: 50 });
test.prop([arbBatch])('one feature per MMSI that has a position', (reports) => {
  const expected = new Set(reports.filter(hasPosition).map((r) => r.mmsi)).size;
  const fcol = vesselsToCollection(reports);
  expect(fcol.features).toHaveLength(expected);
  expectValidGeoJSON(fcol);
});
```

Geometric invariants, with Turf as an independent checker. `coverageToFeature` uses Turf's `circle`. The test checks the result with *different* Turf functions (`booleanPointInPolygon`, `distance`), so it doesn't repeat the implementation:

test/radio.test.ts

```
test.prop([arbSafePosition, arbRange])(
  'contains its station and every vertex is ~rangeKm away',
  (station, rangeKm) => {
    const f = coverageToFeature(station, rangeKm, 'x');
    expectValidGeoJSON(f);
    expect(booleanPointInPolygon(point(station), f)).toBe(true);
    for (const v of f.geometry.coordinates[0]!) {
      expect(distance(station, v, { units: 'kilometers' })).toBeCloseTo(rangeKm, 0);
    }
  },
);
```

Structural rules. Valid GeoJSON can still draw badly. A segment that spans 180° or more of longitude is drawn across the whole map. `fc.pre` documents the known limit at the poles (§07):

test/hf.test.ts

```
// fast-check found this limit: a great circle that passes over (or right next to) a pole flips
// longitude by exactly 180° there. Those paths can't be drawn on Web Mercator (which stops at
// ~85°) anyway, so the property covers paths that stay inside the drawable band.
test.prop([arbHfSpot])('never jumps across the map (no segment spans ≥ 180° of lng)', (s) => {
  const f = hfPathToFeature(s);
  if (!f) return;
  expectValidGeoJSON(f);
  fc.pre(lines(f.geometry).flat().every((p) => Math.abs(p[1]) < 85));
  for (const l of lines(f.geometry)) {
    for (let i = 1; i < l.length; i++) {
      expect(Math.abs(l[i][0]! - l[i - 1][0]!)).toBeLessThan(180);
    }
  }
});
```

Idempotence. Rounding something that is already rounded must not change it:

test/factories.test.ts

```
const arbFinite = fc.double({ noNaN: true, noDefaultInfinity: true, min: -1e6, max: 1e6 });
test.prop([arbFinite])('is idempotent', (x) => {
  expect(round6(round6(x))).toBe(round6(x));
});
```

Round-trip. MapLibre serialises source data to its worker, so a feature must survive JSON. Also, a bearing line drawn with `destination()` must give back its bearing:

docs/examples/fast-check/05-patterns.test.ts

```
// MapLibre serialises source data to its worker; nothing may change on the way.
test.prop([arbAisReport])('an AIS feature survives JSON.stringify -> JSON.parse', (m) => {
  const f = aisToFeature(m);
  expect(JSON.parse(JSON.stringify(f))).toEqual(f);
});

// Out with destination(), back with bearing(): we should get the DF bearing again.
test.prop([
```

```

    arbSafePosition,
    fc.double({ min: 0, max: 359.99, noNaN: true }),
    fc.double({ min: 5, max: 500, noNaN: true }),
  ])(`bearing(start, end) of a DF line is the bearing we drew`, (station, deg, rangeKm) => {
    const line = dfBearingToFeature(station, deg, rangeKm, { stationId: 'df-1' });
    const [start, end] = line.geometry.coordinates as [Position, Position];
    expect(angleBetween(normaliseBearing(bearing(start, end)), deg)).toBeLessThan(0.01);
  });

```

Order-independence. Feeds arrive out of order, so the result must not depend on the order:

test/adsb.test.ts

```

test.prop([arbTrackSamples])(`is order-independent (sorted by timestamp)`, (s) => {
  fc.pre(new Set(s.map((x) => x.t)).size === s.length); // unique timestamps
  expect(adsbTrackToFeature('a', [...s].reverse())).toEqual(adsbTrackToFeature('a', s));
});

```

test/ais.test.ts

```

const ids = (r: AisPositionReport[]) =>
  vesselsToCollection(r).features.map((f) => f.id).sort();
test.prop([fc.array(arbAisReport, { maxLength: 30 })])(
  `order of input does not matter`,
  (reports) => {
    expect(ids([...reports].reverse())).toEqual(ids(reports));
  },
);

```

06 Reading a failure

A failing property prints a report, followed by `Caused by:` and your own assertion error. This is the report for the wrong `deg % 360` property from §01. `fc.check` returns the same report as a value, so this test pins its first four lines:

docs/examples/fast-check/06-reading-a-failure.test.ts

```

const report = [
  'Property failed after 3 tests',
  '{ seed: 42, path: "2:1:0:0:0:0:0:0:0", endOnFailure: true }',
  'Counterexample: [-1]',
  'Shrunk 9 time(s)',
];

```

Line	What it tells you
failed after 3 tests	The third generated input failed. A failure after 1 test usually means the property itself is wrong.
seed: 42	Replays the whole run. <code>test.prop</code> also shows it in the test name.
path: "2:1:0:..."	The failing run (index 2), then each shrink step. Replays straight to the counterexample.

Line	What it tells you
Counterexample: [-1]	The shrunk input, with one entry per arbitrary. Start debugging here.
Shrunk 9 time(s)	How far it got from the first failing input. Pass <code>verbose: true</code> to list them all.

Replay. Run `FC_SEED=42 npm test -- -t "<name>"` to repeat the whole run. To jump straight to the counterexample, pass `{ seed, path }` as parameters, e.g. `test.prop([arb], { seed: 42, path: '2:1:0:0:0:0:0:0:0' })`. Remove them again once the bug is fixed:

`docs/examples/fast-check/06-reading-a-failure.test.ts`

```
it('replays straight to the shrunk counterexample with seed + path', () => {
  const replay = fc.check(neverNegative, { seed: 42, path: '2:1:0:0:0:0:0:0:0' });
  expect(replay.numRuns).toBe(1); // no search, no shrinking: one run
  expect(replay.counterexample).toEqual([-1]);
});
```

Pin it. A seed only replays the failure while the arbitraries stay the same. To keep a counterexample for good, add it to `examples`. Those inputs run first, on every run. Our HF regressions as `examples`:

`docs/examples/fast-check/06-reading-a-failure.test.ts`

```
const spot = (tx: Position, rx: Position): HfSpot => ({
  tx, rx, txCall: 'DL1ABC', rxCall: 'W2XYZ', frequencyKHz: 14074, mode: 'FT8', snrDb: -12,
  timestamp: '2026-10-06T19:30:15Z',
});

test.prop([arbHfSpot], {
  // Run before the random values, on every run of the suite.
  examples: [
    [spot([-30, 10], [180, 0])], // Turf ended this path at +180 (see 07)
    [spot([-180, 0], [179, 0])], // Turf returned a part with a single point
  ],
})('HF paths never jump across the map', (s) => {
  const f = hfPathToFeature(s);
  if (!f) return;
  const g = f.geometry;
  const parts = g.type === 'LineString' ? [g.coordinates] : g.coordinates;
  fc.pre(parts.flat().every(([, lat]) => Math.abs(lat!) < 85)); // known limit at the poles
  for (const part of parts) {
    expect(part.length).toBeGreaterThanOrEqual(2);
    for (let i = 1; i < part.length; i++) {
      expect(Math.abs(part[i][0]! - part[i - 1][0]!)).toBeLessThan(180);
    }
  }
});
```

examples or a separate it? Use `examples` when the counterexample is just another input to the same property. Use a named `it` (what we did in §07) when the case deserves its own name and comment in the test report.

07 Case study: what fast-check found

These came up while the repo was being built. Each one is an input nobody would have written as an example. Each is now a regression test.

1. A path that ends at +180 after approaching from the west

Counterexample: an HF spot from `[-30, 10]` to `[180, 0]`. **Root cause:** `-180` and `+180` are the same meridian, and Turf's `greatCircle` returns the endpoint as given, at `+180`, after approaching it from the west. With few points that gives one `LineString` whose last step goes from about `-170.7` to `+180`. With 64 points (our default) it adds a zero-length part on the far side of the map. Either way, MapLibre draws a line across the whole world:

`docs/examples/fast-check/07-case-study.test.ts`

```
it('raw Turf output jumps across the whole map; ours does not', () => {
  // 16 points: one LineString whose last step goes from about -170.7 to +180
  const coarse = greatCircle([-30, 10], [180, 0], { npoints: 16 }).geometry;
  expect(coarse.type).toBe('LineString');
  expect((coarse.coordinates as Position[]).at(-1)).toEqual([180, 0]);
  expect(maxLngJump([coarse.coordinates as Position[]])).toBeGreaterThan(350);

  // 64 points: Turf splits, but adds a zero-length part on the far side of the map
  const fine = greatCircle([-30, 10], [180, 0], { npoints: 64 }).geometry;
  expect((fine.coordinates as Position[][]).at(-1)).toEqual([[180, 0], [180, 0]]);

  const ours = hfPathToFeature(spot([-30, 10], [180, 0]))!;
  expect((ours.geometry.coordinates as Position[]).at(-1)).toEqual([-180, 0]);
});
```

Fix: move every ± 180 point to the side of its neighbour, before splitting:

`src/geo/hf.ts`

```
/**
 * -180 and 180 are the same meridian. Turf sometimes ends a path at +180 when the line
 * approaches from the west (or vice versa), which MapLibre draws as a line across the whole
 * map. Flip such points to the side of their previous neighbour.
 * (Found by the fast-check tests.)
 */
export function alignAntimeridian(line: Position[]): Position[] {
  const out: Position[] = [];
  line.forEach((p, i) => {
    if (Math.abs(p[0]!) !== 180) return void out.push(p);
    // Side of the previous point (already aligned, so a run of  $\pm 180$  points stays on one side),
    // or, for a leading  $\pm 180$  point, of the first point that isn't on the antimeridian.
    const ref =
      out[out.length - 1]?.[0] ?? line.slice(i + 1).find((q) => Math.abs(q[0]!) !== 180)?.[0];
    out.push(ref === undefined ? p : [(Math.sign(ref) || 1) * 180, ...p.slice(1)]);
  });
  return out;
}
```

Pinned:

test/hf.test.ts

```
// Regression pinned from a fast-check counterexample: Turf ended this path at +180 after
// approaching from the west, which drew a line across the whole map.
it('endpoint exactly on the antimeridian stays on the near side', () => {
  const f = hfPathToFeature(spot([-30, 10], [180, 0]));
  const c = lines(f.geometry)[0];
  expect(c[c.length - 1][0]).toBe(-180);
  for (let i = 1; i < c.length; i++) {
    expect(Math.abs(c[i][0] - c[i - 1][0])).toBeLessThan(180);
  }
});
```

2. A MultiLineString part with a single point

Counterexample: a near-antipodal path whose Turf output, flattened, was `[[ -174.3, 0], [-177.15, 0], [180, 0]]`. **Root cause:** Turf can return a part with one position, and a LineString needs at least two. That is invalid GeoJSON, and the validity property (`expectValidGeoJSON`) rejects it. Here is a short reproducer we found while writing this guide:

docs/examples/fast-check/07-case-study.test.ts

```
it('raw Turf output is invalid GeoJSON; ours is valid', () => {
  const raw = greatCircle([-180, 0], [179, 0], { npoints: 64 }).geometry;
  expect(raw.type).toBe('MultiLineString');
  expect((raw.coordinates as Position[][]).map((part) => part.length)).toEqual([1, 63]);

  const ours = hfPathToFeature(spot([-180, 0], [179, 0]));
  expectValidGeoJSON(ours);
  expect(ours.geometry.type).toBe('LineString');
});
```

Fix and pin: `splitAtAntimeridian` (below) splits exactly at ± 180 , interpolating the crossing latitude, and drops degenerate parts:

test/hf.test.ts

```
it('drops single-point parts (Turf produced one for a near-antipodal path)', () => {
  const parts = splitAtAntimeridian([[-174.3, 0], [-177.15, 0], [180, 0]]);
  expect(parts).toEqual([[-174.3, 0], [-177.15, 0], [-180, 0]]);
});
```

3. Our own fix divided 0 by 0

Counterexample: an over-the-pole path with consecutive points on the antimeridian: `[[0, -89.03], [-180, -88.13], [-180, -85.29], [-180, -82.44]]`. **Root cause:** when one point is written as +180 and the next as -180, the code saw an antimeridian crossing of zero width, and the interpolation became $0 / 0$. That gave `NaN`, and `NaN` serialises to `null` in JSON. **Fix:** the `bUnwrapped` `!== a[0]` guard below:

src/geo/hf.ts

```
/**
 * Normalise a path for Web Mercator rendering:
 * - one ordered list of points in, any number of parts out
 * - split wherever consecutive points jump more than 180° of longitude (an antimeridian
 *   crossing), interpolating the crossing latitude so each part ends exactly at  $\pm 180$  (no gap)
```

```

* - drop degenerate parts (fewer than 2 distinct positions), which are invalid GeoJSON
*
* Turf's greatCircle output is fed through this because fast-check found both a wrong-side
* ±180 endpoint and a MultiLineString part containing a single point.
*/
export function splitAtAntimeridian(points: Position[]): Position[][] {
  const pts = alignAntimeridian(points.map(roundPosition));
  if (pts.length === 0) return [];
  const parts: Position[][] = [[pts[0]!]];
  const pushTo = (part: Position[], p: Position) => {
    if (!samePos(part[part.length - 1]!, p)) part.push(p);
  };

  for (let i = 1; i < pts.length; i++) {
    const a = pts[i - 1]!;
    const b = pts[i]!;
    const current = parts[parts.length - 1]!;
    const side = Math.sign(a[0]!) || 1; // +1: leaving eastwards over +180
    const bUnwrapped = b[0]! + side * 360;
    if (Math.abs(b[0]! - a[0]!) > 180 && bUnwrapped !== a[0]!) {
      const t = (side * 180 - a[0]!) / (bUnwrapped - a[0]!);
      const lat = round6(a[1]! + t * (b[1]! - a[1]!));
      pushTo(current, [side * 180, lat]);
      parts.push([[-side * 180, lat]]);
      pushTo(parts[parts.length - 1]!, b);
    } else {
      pushTo(current, b);
    }
  }
  return parts.filter((part) => part.length >= 2);
}

```

test/hf.test.ts

```

// Regression from a fast-check counterexample: an over-the-pole path has consecutive points
// on the antimeridian; interpolating between them divided 0 by 0 and emitted null latitudes.
it('never emits NaN for consecutive antimeridian points (over-the-pole path)', () => {
  const overThePole = [[0, -89.03], [-180, -88.13], [-180, -85.29], [-180, -82.44]];
  const parts = splitAtAntimeridian(overThePole);
  for (const [lng, lat] of parts.flat()) {
    expect(Number.isFinite(lng) && Number.isFinite(lat)).toBe(true);
  }
});

```

Known limits, documented instead of fixed

A great circle over a pole flips longitude by 180° at the pole. Web Mercator stops at about 85°, so the property limits itself to paths it can draw (the `fc.pre` in §05). The limit is pinned so we notice if Turf changes:

docs/examples/fast-check/07-case-study.test.ts

```

it('a great circle over the pole flips longitude by 180°', () => {
  const coords = greatCircle([0, 60], [180, 60], { npoints: 7 }).geometry.coordinates;
  expect(coords).toEqual([
    [0, 60], [0, 70], [0, 80], [90, 90], [180, 80], [180, 70], [180, 60],
  ]);
});

```

Coverage circles near ± 180 keep a continuous ring (longitudes like 180.3). MapLibre draws that without a seam. The Turf guide covers what this means for point-in-polygon checks.

08 Pitfalls

Pitfall	Symptom	Fix
Restating the implementation	The test passes and the bug ships, because the test has the same bug	Assert a property of the output, not the formula
Too much <code>filter</code> / <code>fc.pre</code>	Slow runs, or "too many pre-condition failures"	Build the constraint into the arbitrary
Comparing floats with <code>toBe</code>	Fails on rounding noise	<code>toBeCloseTo(x, digits)</code> , i.e. difference $< 10^{-\text{digits}/2}$
NaN, $\pm$Infinity, -0	Surprises in equality and in JSON	<code>noNaN</code> , <code>finite</code> <code>min/max</code> , <code>normalise -0</code>
Slow properties	The suite takes minutes	Fewer runs per test, smaller <code>maxLength</code>
<code>return expect(...)</code>	"Property failed by returning false" although every assertion passed	Use a block body <code>{ ... }</code> , so nothing is returned

Restating the implementation. If the test computes the expected value the same way the code does, it can only find typos. Describe what is true about the result instead:

docs/examples/fast-check/08-pitfalls.test.ts

```
// ✗ Avoid: a copy of the implementation. If the formula is wrong, both are wrong.
test.prop([arbDeg])('mirror test', (deg) => {
  expect(normaliseBearing(deg)).toBe(((deg % 360) + 360) % 360 || 0);
});

// ✓ Prefer: what must be true, without saying how to compute it.
test.prop([arbDeg])('in range, and a whole number of turns away from the input', (deg) => {
  const b = normaliseBearing(deg);
  expect(b >= 0 && b < 360).toBe(true);
  const turns = (deg - b) / 360;
  expect(Math.abs(turns - Math.round(turns))).toBeLessThan(1e-6);
});
```

filter vs fc.pre vs a constrained arbitrary. All three give unique timestamps. The first two throw away the inputs that don't fit, and with a strict condition most of the runs are wasted. fast-check gives up with "too many pre-condition failures" when too many runs are skipped. The third wastes nothing:

docs/examples/fast-check/08-pitfalls.test.ts

```
// 1. filter: generated values that fail the predicate are thrown away and regenerated.
const viaFilter = fc.array(arbSample, { maxLength: 40 }).filter(uniqueTimes);

// 2. fc.pre: the run is skipped (counted in numSkips); too many skips fail the test.
const withPre = fc.property(fc.array(arbSample, { maxLength: 40 }), (samples) => {
  fc.pre(uniqueTimes(samples));
});
```

```
});

// 3. Constrained: unique by construction, nothing is thrown away.
const viaUnique = fc.uniqueArray(arbSample, { maxLength: 40, selector: (s) => s.t });
```

Comparing floats. `toBeCloseTo(x, 5)` passes when the difference is below 0.000005. Pick the number of digits from the domain: 6 decimal places of a degree is about 10 cm.

docs/examples/fast-check/08-pitfalls.test.ts

```
test.prop([arbLng])('round6 moves a longitude by less than 0.000005°', (lng) => {
  // expect(round6(lng)).toBe(lng) fails for almost every input
  expect(round6(lng)).toBeCloseTo(lng, 5); // |difference| < 0.000005
});
```

NaN, Infinity and -0. `fc.double()` without constraints generates all of them. Real feeds send them too, so include them on purpose (as `arbGarbageNumber` does) rather than by accident:

docs/examples/fast-check/08-pitfalls.test.ts

```
it('fc.double() without constraints generates NaN, ±Infinity and -0', () => {
  const xs = fc.sample(fc.double(), { seed: 1, numRuns: 1000 });
  expect(xs.some(Number.isNaN)).toBe(true);
  expect(xs).toContain(Number.POSITIVE_INFINITY);
  expect(xs.some((x) => Object.is(x, -0))).toBe(true);
});

it('Vitest treats -0 and 0 as different; JSON does not', () => {
  expect(() => expect(-0).toEqual(0)).toThrow(); // toBe and toEqual use Object.is for numbers
  expect(JSON.parse(JSON.stringify(-0))).toBe(0); // the sign is lost in GeoJSON files
  expect(Object.is(round6(-0.0000001), 0)).toBe(true); // so our factories normalise it
});
```

Slow properties. Each property runs 100 times, so a 20 ms property takes 2 s. Lower `numRuns` for that test (\$02), limit `maxLength` on arrays, and set `interruptAfterTimeLimit` if a property has to stop after a fixed time. Run with `FC_RUNS=2000` when you want more coverage, not on every save.

Never return `expect(...)` from a property. This caused false failures for us. An arrow function with an expression body returns the assertion object. fast-check treats any return value other than `true` or `undefined` as a failure, and reports "Property failed by returning false" even though the assertion passed. TypeScript doesn't catch it, because Vitest types its matchers as returning `void`.

docs/examples/fast-check/08-pitfalls.test.ts

```
it('a returned assertion fails the property, even though the assertion passed', () => {
  // An arrow function with an expression body *returns* the assertion object.
  // fast-check only accepts true or undefined, so this reports
  // "Property failed by returning false".
  const returnsAssertion = fc.property(fc.integer(), (n) => expect(n).toBe(n));
  expect(fc.check(returnsAssertion).failed).toBe(true);

  // Fix: use a block body, so nothing is returned.
  const fixed = fc.property(fc.integer(), (n) => {
```

```
    expect(n).toBe(n);
  });
  expect(fc.check(fixed).failed).toBe(false);
});
```

09 Advanced: model-based testing

Some code has state: a store that keeps the latest position per MMSI and drops contacts nobody has heard from in 10 minutes. Its bugs show up in *sequences* of operations, such as an expiry exactly 10 minutes after a report, or two reports with the same timestamp. **Model-based testing** generates such sequences. It runs each one on the real code and on a much simpler **model**, and checks after every step that the two agree.

The system under test (docs/examples/fast-check/live-feed-store.ts):

docs/examples/fast-check/live-feed-store.ts

```
export class LiveFeedStore {
  protected readonly latest = new Map<number, VesselFeature>();

  constructor(private readonly maxAgeMs: number) {}

  /** Keeps the report if it has a position and is not older than what we have. */
  ingest(report: AisPositionReport): void {
    const f = aisToFeature(report);
    if (!f) return;
    const prev = this.latest.get(f.properties.mmsi);
    if (!prev || f.properties.timestamp >= prev.properties.timestamp) {
      this.latest.set(f.properties.mmsi, f);
    }
  }

  /** Drops contacts whose last report is older than maxAgeMs at `nowMs`. */
  expire(nowMs: number): void {
    for (const [mmsi, f] of this.latest) {
      if (nowMs - Date.parse(f.properties.timestamp) > this.maxAgeMs) this.latest.delete(mmsi);
    }
  }

  get(mmsi: number): VesselFeature | undefined {
    return this.latest.get(mmsi);
  }

  get size(): number {
    return this.latest.size;
  }

  toCollection(): FeatureCollection<Point, VesselProps> {
    return { type: 'FeatureCollection', features: [...this.latest.values()] };
  }
}
```

The model is a `Map` from MMSI to time and position. Each **command** updates the model, runs the real operation and compares the two. `check` decides whether a command can run in the current state (all of ours always can):

docs/examples/fast-check/09-model-based.test.ts

```
/** The model: just "MMSI -> time and position of the newest kept report". */
type Contact = { t: number; lng: number; lat: number };
type Model = { contacts: Map<number, Contact> };
type Real = LiveFeedStore;

const hasPosition = (r: AisPositionReport) =>
  r.lat !== 91 && r.lon !== 181 && Math.abs(r.lat) <= 90 && Math.abs(r.lon) <= 180;

/** After every command the store must hold exactly the model's contacts. */
function expectSameState(m: Model, r: Real) {
  expect(r.size).toBe(m.contacts.size);
  for (const [mmsi, c] of m.contacts) {
    const f = r.get(mmsi)!;
    expect(Date.parse(f.properties.timestamp)).toBe(c.t);
    expect(f.geometry.coordinates).toEqual([round6(c.lng), round6(c.lat)]);
  }
}
```

docs/examples/fast-check/09-model-based.test.ts

```
class IngestCommand implements fc.Command<Model, Real> {
  constructor(readonly report: AisPositionReport) {}
  check = () => true; // always allowed
  run(m: Model, r: Real) {
    const { mmsi, lon, lat, timestamp } = this.report;
    const t = Date.parse(timestamp);
    const prev = m.contacts.get(mmsi);
    if (hasPosition(this.report) && (prev === undefined || t >= prev.t)) {
      m.contacts.set(mmsi, { t, lng: lon, lat }); // a newer or equally new report wins
    }
    r.ingest(this.report);
    expectSameState(m, r);
  }
  toString = () => `ingest(${this.report.mmsi} @ ${this.report.timestamp})`;
}

class ExpireCommand implements fc.Command<Model, Real> {
  constructor(readonly nowMs: number) {}
  check = () => true;
  run(m: Model, r: Real) {
    for (const [mmsi, c] of m.contacts) {
      if (this.nowMs - c.t > MAX_AGE_MS) m.contacts.delete(mmsi);
    }
    r.expire(this.nowMs);
    expectSameState(m, r);
    expectValidGeoJSON(r.toCollection());
  }
  toString = () => `expire(T0 + ${(this.nowMs - T0) / 60_000} min)`;
}
```

`fc.commands` generates the sequences and shrinks them by removing commands. Keep the pool of MMSIs and times small, so reports collide and overtake each other:

`docs/examples/fast-check/09-model-based.test.ts`

```
// Few MMSIs and a short time window, so reports for the same vessel collide and overtake.
const arbMinutes = fc.integer({ min: 0, max: 60 });
const arbReport = fc
  .tuple(arbAisReport, fc.constantFrom(211_000_001, 211_000_002, 211_000_003), arbMinutes)
  .map(([r, mmsi, min]: AisPositionReport => ({
    ...r,
    mmsi,
    timestamp: new Date(T0 + min * 60_000).toISOString(),
  })));

const arbCommands = fc.commands(
  [
    arbReport.map((r) => new IngestCommand(r)),
    arbMinutes.map((min) => new ExpireCommand(T0 + min * 60_000)),
  ],
  { maxCommands: 50 },
);

const setup = (real: LiveFeedStore) => () => ({ model: { contacts: new Map() }, real });

test.prop([arbCommands])('LiveFeedStore behaves like the model', (cmds) => {
  fc.modelRun(setup(new LiveFeedStore(MAX_AGE_MS)), cmds);
});
```

The off-by-one we would write by accident: a store that expires a contact when it is *exactly* 10 minutes old. fast-check finds it and shrinks the failure to two commands: a report at 19:17 and an expiry at 19:27.

`docs/examples/fast-check/09-model-based.test.ts`

```
/** A store with an off-by-one: it expires contacts that are exactly MAX_AGE_MS old. */
class OffByOneStore extends LiveFeedStore {
  override expire(nowMs: number): void {
    for (const [mmsi, f] of this.latest) {
      if (nowMs - Date.parse(f.properties.timestamp) >= MAX_AGE_MS) this.latest.delete(mmsi);
    }
  }
}

it('finds the off-by-one and shrinks it to two commands', () => {
  const details = fc.check(
    fc.property(arbCommands, (cmds) => {
      fc.modelRun(setup(new OffByOneStore(MAX_AGE_MS)), cmds);
    }),
    { seed: 9 },
  );
  // A report at 19:17 and an expiry exactly 10 minutes later: the boundary case.
  expect(String(details.counterexample![0])).toBe(
    'ingest(211000001 @ 2026-10-06T19:17:00.000Z),expire(T0 + 27 min) /*replayPath="DCW:F"*/',
  );
});
```

To replay a sequence, use the seed and path from the report, and pass the printed `replayPath` to `fc.commands(..., { replayPath })`. Use `fc.asyncModelRun` and `fc.AsyncCommand` when the real system is async.

10 Exercises and bookmarks

1. **Bands.** Write properties for `bandForKHz`. Every frequency from 14,000 to 14,350 kHz is `'20m'`, and frequencies between the bands are `'unknown'`. Generate the gaps with `fc.oneof` over ranges, not with `filter`.
2. **Emergencies.** Property: `emergency` is true exactly when the squawk is 7500, 7600 or 7700. `arbReadsbAircraft` almost never generates those codes. Check that with `fc.sample`, then fix it with a weighted `oneof`.
3. **Tracks.** For `adsbTrackToFeature`: `startTime <= endTime`, and `pointCount` is never more than the number of valid samples.
4. **Geofences.** `featuresInZone` returns a subset of its input in the same order, and it is idempotent: applying it to its own output changes nothing.
5. **Break it on purpose.** In `live-feed-store.ts`, change `>=` to `>` in `ingest`. Run the model test, read the counterexample and explain it. Then undo the change.
6. **Time it.** Run `FC_RUNS=5000 npm test`, find the slowest property in the Vitest output, and make it faster without lowering its run count.

Bookmarks

- fast-check documentation, start here: `fast-check.dev/docs/introduction`
- Every arbitrary, with its options: `fast-check.dev/docs/core-blocks/arbitraries`
- Runners and parameters (`numRuns`, `seed`, `path`, `examples`): `fast-check.dev/docs/core-blocks/runners`
- Model-based testing: `fast-check.dev/docs/advanced/model-based-testing`
- Migrating from 3.x to 4.x: `fast-check.dev/docs/migration-guide/from-3.x-to-4.x`
- @fast-check/vitest: `github.com/dubzzz/fast-check/tree/main/packages/vitest`

Sources

- API details checked against the installed packages: `node_modules/fast-check/lib/fast-check.d.ts` (4.10.2) and `node_modules/@fast-check/vitest/lib/vitest-fast-check.js` (0.5.0)
- fast-check 3 → 4 changes: the migration guide above, and the type definitions of fast-check 3.23.2 compared with 4.10.2
- AIS "not available" values: ITU-R M.1371 — `itu.int/rec/R-REC-M.1371`
- Vitest `toBeCloseTo` and other matchers — `vitest.dev/api/expect.html`
- The bugs in §07: this repo's README, section "Bugs fast-check found while building this", and `test/hf.test.ts`